SK네트웍스 **Family AI**과정 **10**기

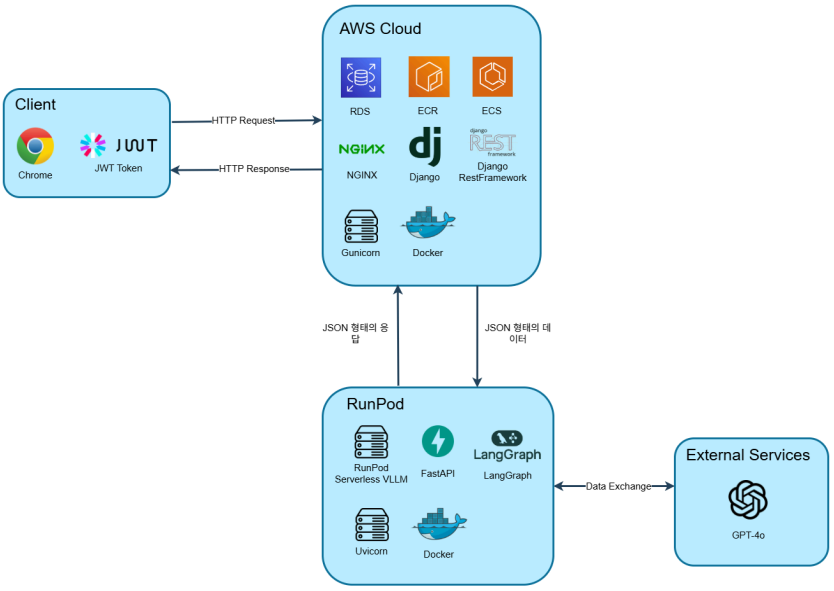
모델링 및 평가 시스템 아키텍처

□ 개요

- 산출물 단계 : 모델링 및 평가
- 평가 산출물 : 시스템 아키텍처
- 제출 일자 : 2025. 06. 27.
- 깃허브 경로 : https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN10-FINAL-3Team
- 작성 팀원 : 문승기, 김현수

컴포넌트 다이어그램	• 구성 요소 • 설명
시퀀스 다이어그램	• 참여자 (Actors) • 인프라 및 배포 • 주요 흐름
액티비티 다이어그램	• 구성 • 주요 액션 노드

1. 컴포넌트 다이어그램



구성 요소

1. **클라이언트(Client):**
   - 컴포넌트
     - 크롬 브라우저
   - 인터페이스
     - JWT 토큰 (인증·인가)
     - JSON 데이터 전송 (HTTPS)
2. **AWS Cloud:**
   - ECR (Elastic Container Registry):
     - Docker : 애플리케이션 실행 환경을 컨테이너화
     - Nginx : 리버스 프록시 서버 및 정적 파일 제공
     - Gunicorn : Django 앱 실행을 위한 WSGI 서버
     - Django : 주요 웹 로직을 담당하는 백엔드 프레임워크
     - Django Rest Framework : API 서버 구현
   - ECS (Elastic Container Service):
     - 웹 애플리케이션을 무중단 배포
     - 헬스체크 및 자동 재시작
     - GitHub Actions 등 CI 파이프라인과 연동하여 자동화 배포 구성
   - RDS (Relational Database Service):
     - 주요 저장 데이터:
       - 구조화된 정형 데이터
       - 사용자 정보, 사건 데이터, 판례 데이터 등

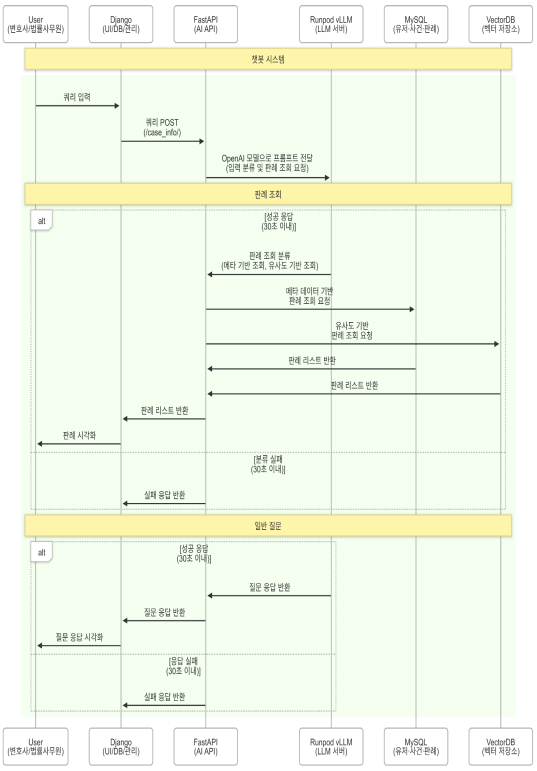
3. **RunPod:**
   - RunPod Serverless vLLM
   - Docker : 애플리케이션 실행 환경을 컨테이너화
     - 이미지 구성:
       - FastAPI : JSON 형태의 요청을 받아 모델 응답 반환
       - Uvicorn : FastAPI 서버를 실행하는 ASGI 서버

4. **외부 서비스:**
   - ChatGPT (OpenAI 언어 모델)

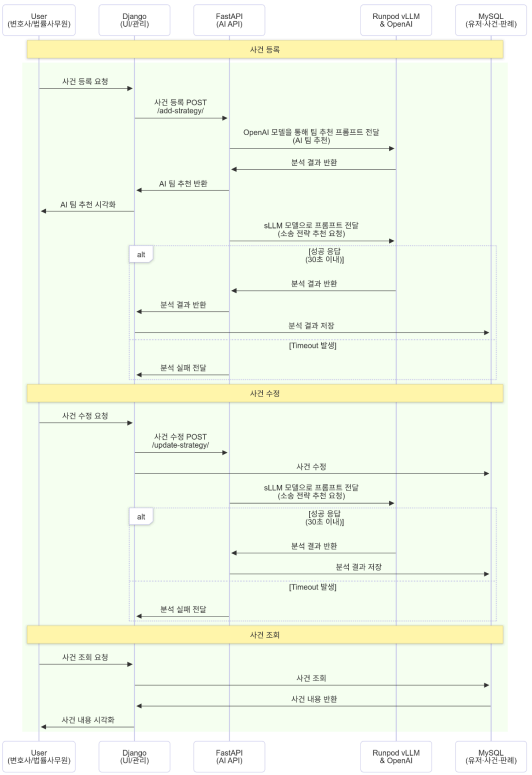
설명

1. **클라이언트**
   - Chrome 브라우저를 통해 서비스에 접속
   - 로그인 시 발급된 **JWT 토큰을 쿠키에 저장**하고 인증/인가를 수행
   - 사용자의 입력 또는 요청을 **JSON 포맷으로 변환**
   - **HTTP 프로토콜을 통해 AWS Cloud로 전송**.
2. **AWS Cloud**
   - GitHub Actions를 통해 애플리케이션 코드를 자동으로 빌드
   - Docker 이미지로 패키징하여 **ECR**에 **저장**
   - **ECS**를 통해 해당 **이미지**를 **실행** 및 **관리**
   - 컨테이너 내부에서는 Nginx를 통해 요청을 수행
   - Gunicorn을 통해 Django 애플리케이션을 실행
   - **Django REST Framework**를 통해 **API 요청**을 **처리**
   - **RDS(MySQL)**와 **연결**하여 **데이터 조회** 및 **저장**
   - **AI 관련 요청**은 **RunPod**(FastAPI)로 전달
3. **Runpod**
   - FastAPI는 Agent를 통해 ChatGPT(GPT-4o)를 호출하여 분기를 나눔
   - 나눠진 분기에 따라 함수를 호출하여 응답을 생성하거나
     파인튜닝된 모델에 요청을 전달
   - **Serverless vLLM**을 활용해 **추론 결과**를 **생성**.
   - 모든 처리 결과는 JSON 형태로 반환
     AWS Cloud를 거쳐 클라이언트에 전달
     사용자는 웹 화면에서 결과를 확인

2. 시퀀스 다이어그램



챗봇 시퀀스 다이어그램



사건 등록 시퀀스 다이어그램

참여자

1. **User:** 웹 애플리케이션을 통해 요청을 시작하는 사용자(변호사, 법률 사무원)
2. **Django:**
   - JWT 토큰을 통해 인증·인가 처리
   - 클라이언트 요청을 수신 및 FastAPI로 전달
3. **FastAPI:**
   - Django 서버로부터 요청을 수신하여
     Runpod serverless vLLM 또는 OpenAI를 호출 후 응답 반환
4. **RunPod vLLM, OpenAI:**
   - FastAPI 요청을 통해
     파인튜닝한 모델(전략 생성) 또는 OpenAI 모델(챗봇)이 응답 생성
5. **MySQL:**
   - 관계형 데이터베이스 작업 수행
6. **VectorDB:**
   - Vector DB 유사도 검색 수행

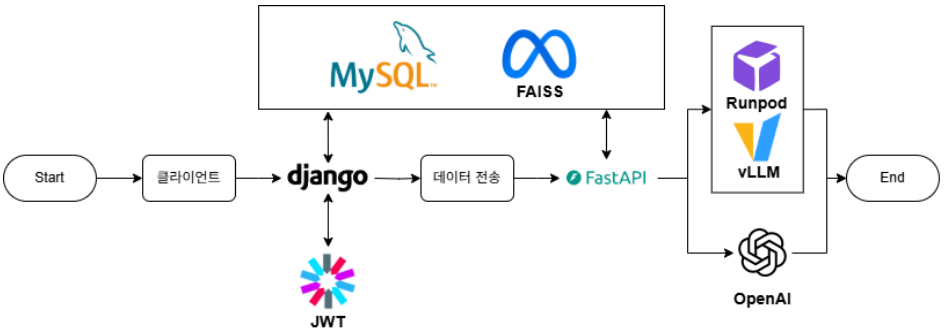
인프라 및 배포

1. **Django 서버**
   - AWS 배포
   - Gunicorn + Nginx
   - JWT 인증·인가 방식 적용
2. **FastAPI 서버**
   - Runpod 배포
   - Uvicorn
3. **DB**
   - MySQL: AWS RDS 배포
   - FAISS: FastAPI 내부 저장
4. **LLM 모델**
   - Runpod serverless vLLM 인스턴스 실행
     - Gemma-3-4B-it 파인튜닝 모델
   - OpenAI GPT 4.1 모델

주요 흐름

1. 사용자가 클라이언트를 통해 요청을 시작.
2. Django에서 JWT 토큰을 기반으로 사용자 인증·인가
3. Django에서 사용자의 요청을 FastAPI로 전달
4. FastAPI는
   1) 소송 전략 생성일 때:
      Runpod vLLM 모델에 프롬프트 전달하여 전략 생성 및 반환
   2) 챗봇 입력일 때:
      OpenAI 모델에 프롬프트 전달하여 질문에 대한 응답 생성 및 반환
5. FastAPI가 모델이 반환한 결과를 Django로 전달
6. 사용자는 Django를 통해 결과를 확인.

3. 액티비티 다이어그램



구성

1. **Start**
   - 시스템 동작
2. **클라이언트 활동**
   - 사용자가 웹 애플리케이션을 통해 **요청**을 보냄
   - 사건 등록, 판례 검색 등
3. **Django**
   - **JWT 인증·인가** 처리
   - 사용자의 요청을 받아 FastAPI로 데이터 전송
   - DB를 통해 **정보 조회·수정**
4. **인증·인가 (JWT)**
   - Django 미들웨어를 통해 JWT 인증 처리
   - **토큰 검증** 및 **권한 부여**
5. **데이터 전송**
   - Django → FastAPI로 요청 전송
6. **FastAPI**
   - 전달받은 데이터를 기반으로 프롬프트를 구성하여 모델에 전달
   - 처리 결과를 **응답 형태**로 **반환** 및 **DB 저장**
7. **Runpod vLLM / OpenAI**
   - 사건 전략 추천
   - 메타 또는 유사도 기반 판례 조회
   - 질의 응답
8. **데이터베이스 조회 (MySQL, FAISS)**
   - FastAPI 또는 Django에서 결과 저장 또는 판례, 사건 정보 조회 수행
9. **End**
   - 클라이언트에 결과 응답 반환

주요 액션 노드

- **사건 등록:**
  - Django → FastAPI: “/add-strategy/” POST 요청
  - 전략 추천(vLLM) + 팀 추천
  - 결과 저장(MySQL)

- **사건 수정:**
  - Django → FastAPI: “/update-strategy/” POST 요청
  - 전략 재추천(vLLM) → 수정된 전략 저장(MySQL)

- **사건 조회:**
  - Django가 직접 MySQL에서 사건 정보 조회

- **챗봇 질문:**
  - Django → FastAPI: “/combined/” POST 요청
  - 과정
    - 프롬프트 구성
      → OpenAI 모델 전달
      → 메타 기반 또는 유사도 기반 판례 검색
  - 검색 결과 반환 → Django 시각화